

GN2: Classificazione del Jet Flavour tramite Transformer

Luca Nasini

20 maggio 2026

1 Motivazione

I jet adronici sono gli oggetti più abbondanti nelle collisioni protone-protone all'LHC. Identificare il sapore di un jet (se origina da un quark b , c , un leptone τ a decadimento adronico, o da un quark leggero/gluone) è fondamentale per la fisica del Modello Standard.

Gli algoritmi tradizionali seguono un approccio a due stadi: ricostruzione dei vertici spostati con strumenti specializzati, poi un classificatore multivariato ad alto livello (e.g. DL1d). GN2 sostituisce questa pipeline con un unico transformer *end-to-end* che elabora direttamente le tracce, raggiungendo miglioramenti di un fattore $1.5 - 4$ rispetto a DL1d.

2 Dataset e Preprocessing (preprocess.py, dataset.py)

I dati consistono in 5.6 milioni di jet da eventi simulati Monte Carlo $t\bar{t}$ (ATLAS, formato HDF5), con variabili di jet (p_T , η) e 19 variabili di traccia (d_0 , z_0 , significanze, variabili angolari, hit nei rivelatori). Le classi sono sbilanciate (Fig. 1b): i light-jet sono i più abbondanti, mentre c-jet e τ -jet sono sottorappresentati.

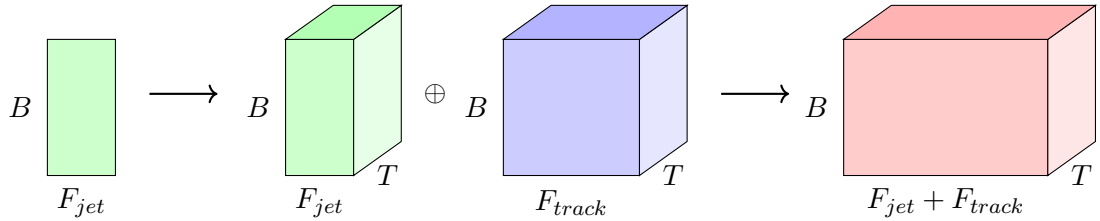
La fase di preprocessing (`preprocess.py`) applica tagli di selezione cinematica ($20 < p_T < 250$ GeV, $|\eta| < 2.5$), e suddivide i dati in training, validazione e test (con proporzioni: 70/20/10%) e la suddivisione avviene per indice casuale. Le statistiche di normalizzazione (media e deviazione standard, con trasformazione logaritmica su p_T) sono calcolate *esclusivamente sul training set* per evitare data leakage. Gli indici di splitting e le statistiche vengono salvati in `XXX_indices.npy` e `norm_stats.json` e riutilizzati durante l'inferenza.

Il caricamento efficiente dei dati è gestito tramite tre componenti: `GN2Dataset` (padding/crop a 40 tracce e normalizzazione), `_IndexDataset` (accessi ordinati su disco) e `_BatchCollator` (letture HDF5 contigue per batch).

3 Architettura del Modello (model.py)

Il modello GN2 (classe `GN2`) elabora ciascun jet in quattro stadi sequenziali:

1. **Inizializzatore per-traccia:** le variabili di jet vengono replicate e concatenate a quelle di traccia; un MLP a due strati mappa il vettore combinato in uno spazio di dimensione 64.



2. **Encoder transformer:** quattro strati con 8 teste di attenzione, embedding di dimensione 64 e feed-forward di dimensione 256, con pre-LayerNorm e connessioni residue; le posizioni di padding sono escluse tramite maschera booleana.
3. **Attention pooling:** una query lineare appresa assegna pesi softmax a ciascun embedding di traccia, aggregandoli in una rappresentazione globale del jet di dimensione 128; questa scelta, rispetto al mean pooling, permette al modello di pesare esplicitamente le tracce più informative (e.g. quelle con parametri d'impatto più significativi).
4. **Testa di classificazione:** un MLP ($128 \rightarrow 64 \rightarrow 32 \rightarrow 4$) produce le probabilità softmax per le quattro classi.

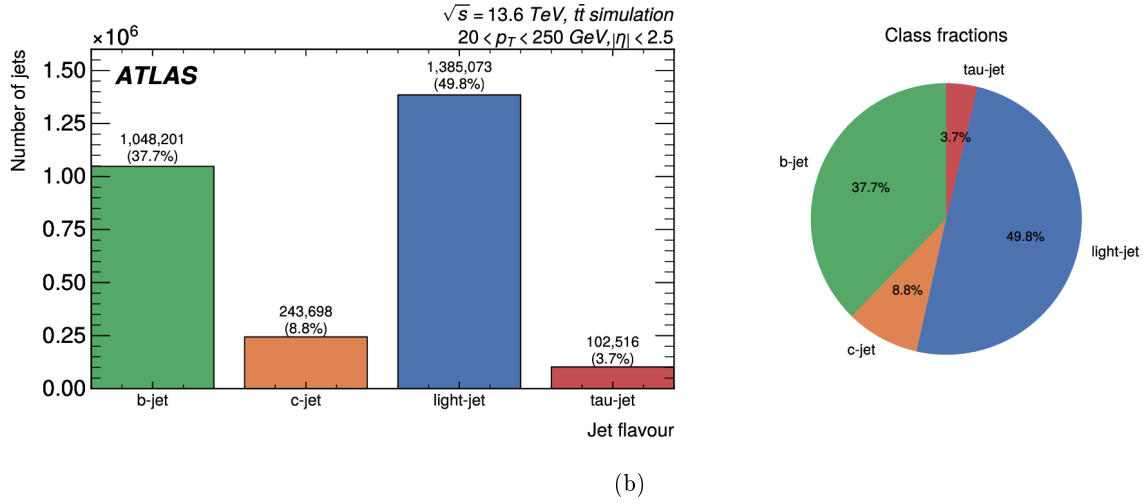
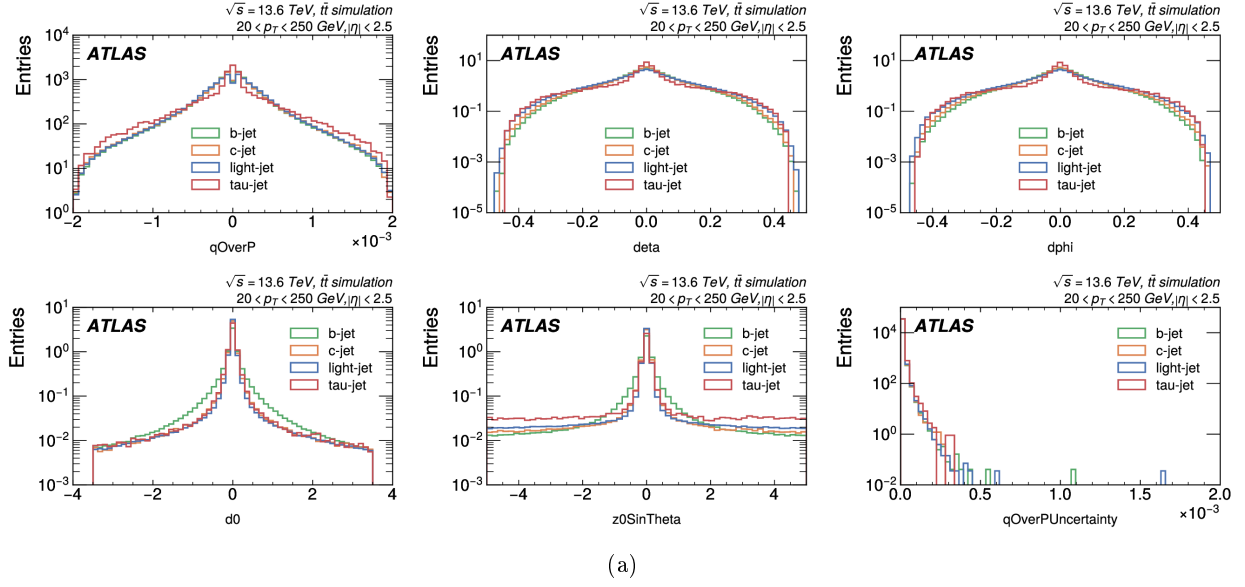


Figura 1: Distribuzioni di alcune variabili di traccia (a) e distribuzione del sapore dei jet nel dataset (b).

Il modello contiene circa **230 000** parametri addestrabili. Le funzioni di attivazione disponibili sono ReLU, LeakyReLU, Sigmoid, Tanh e SoftPlus, con una scelta unica per tutto il modello; è inoltre possibile attivare il dropout.

4 Addestramento (train.py)

Il training è interamente eseguito dalla funzione `train`, che inizializza:

- **Loss:** cross-entropy a quattro classi per il task di classificazione del sapore del jet. I jet con etichetta `-1` vengono ignorati tramite `ignore_index=-1`.
- **Ottimizzatore:** tramite la funzione `get_optimizer` è possibile scegliere tra AdamW, Adam e SGD.
- **Learning rate schedule:** warmup lineare da 10^{-7} a 5×10^{-4} nel primo 1% degli step, seguito da cosine annealing fino a 10^{-5} .

Il ciclo di training è gestito da `run_epoch`, lanciata sia per il training che per la validazione; nel solo caso di training esegue anche il backward pass. Su GPU è attivata la mixed precision automatica (AMP); la norma del gradiente è tagliata a 1.0 per evitare instabilità. Il miglior checkpoint per validation loss viene salvato su disco.

I principali iperparametri dell'addestramento sono riassunti nella Tabella 1.

Iperparametro	Valore
Numero di epoche	21
Dimensione del batch	2048
Numero massimo di tracce	40
Funzione di attivazione	ReLU
Dropout	No
Ottimizzatore	AdamW
Learning rate iniziale	10^{-7}
Learning rate di picco	5×10^{-4}
Learning rate finale	10^{-5}
Weight decay	10^{-5}
Dispositivo	GPU (Tesla T4)
Tempo di training	circa 17 ore

Tabella 1: Iperparametri usati nell'addestramento finale.

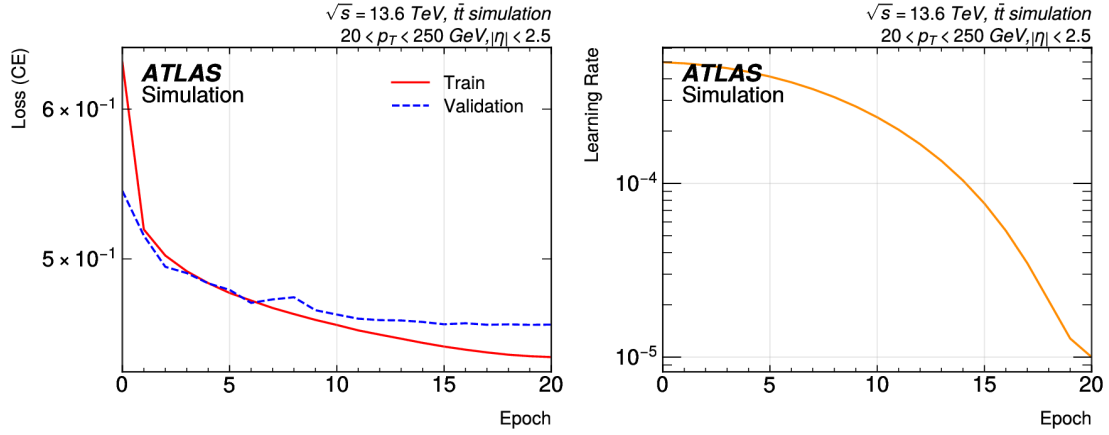


Figura 2: Learning curves (training e validation loss) e profilo del learning rate schedule durante l'addestramento.

5 Valutazione (evaluate.py)

In fase di valutazione il miglior checkpoint viene caricato e applicato ai dati di test. Vengono calcolate le metriche di classificazione e prodotte: la confusion matrix normalizzata, le distribuzioni degli score softmax e le curve ROC per i discriminanti di b - e c -tagging, definiti come:

$$D_b = \log\left(\frac{p_b}{f_c p_c + f_\tau p_\tau + (1 - f_c - f_\tau) p_u}\right), \quad D_c = \log\left(\frac{p_c}{f_b p_b + f_\tau p_\tau + (1 - f_b - f_\tau) p_u}\right).$$

La pipeline produce i seguenti file di output:

- **metrics.json**: accuratezza complessiva e, per ogni classe, precisione, recall e F1-score;
- **confusion_matrix.pdf**: confusion matrix normalizzata (Fig. 4);
- **score_distributions.pdf**: distribuzioni degli score softmax suddivise per sapore reale del jet (Fig. 3);
- **roc_db.pdf**: curve ROC (rigetto del fondo vs. efficienza di b -jet) per il discriminante D_b con $f_c = 0.20$, $f_\tau = 0.05$ (Fig. 5);
- **roc_dc.pdf**: curve ROC analoghe per il discriminante D_c con $f_b = 0.30$, $f_\tau = 0.01$ (Fig. 6).

6 Risultati

La valutazione delle prestazioni del modello viene condotta esclusivamente sul campione di test. Le metriche globali ottenute sono riportate in Tabella 2.

Classe	Precisione	Recall	F1-score	Accuratezza globale
b-jet	0.88	0.86	0.88	0.84
c-jet	0.54	0.16	0.24	
light-jet	0.83	0.96	0.89	
τ -jet	0.69	0.58	0.63	

Tabella 2: Metriche di classificazione sul set di test. I valori si trovano nel file `metrics.json`.

6.1 Distribuzioni degli score di classificazione

Le distribuzioni delle probabilità softmax $P(\text{classe})$ per ciascuna delle quattro categorie sono mostrate in Fig. 3. La distribuzione $P(\text{b-jet})$ mostra una separazione netta: i b-jet veri producono un picco pronunciato vicino a 1, mentre i light-jet dominano la regione a bassa probabilità. Analoga separazione si osserva per $P(\text{light-jet})$. Le distribuzioni $P(\text{c-jet})$ e $P(\tau\text{-jet})$ presentano una sovrapposizione maggiore tra le diverse classi, anticipando le difficoltà di classificazione di queste categorie.

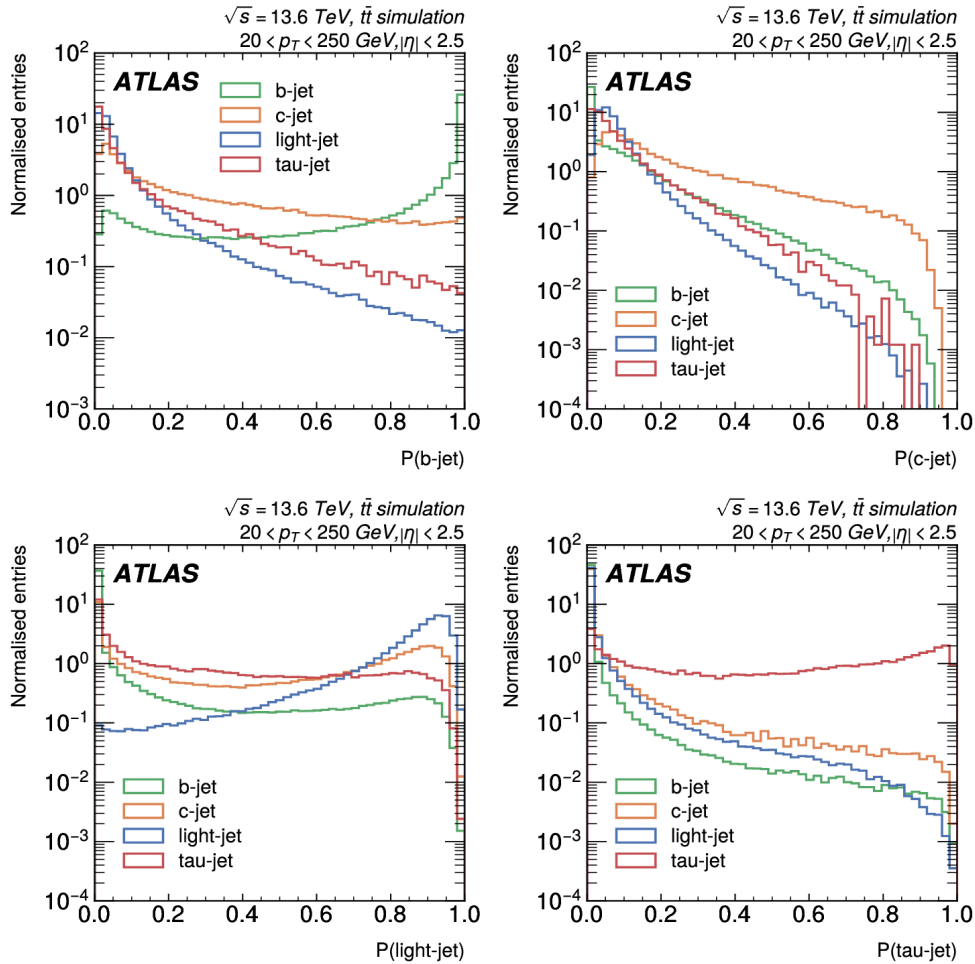


Figura 3: Distribuzioni normalizzate degli score softmax $P(\text{classe})$ per ciascuna delle quattro categorie, separate per sapore reale del jet.

6.2 Matrice di confusione

La matrice di confusione normalizzata (Fig. 4) riporta le probabilità di classificazione mediate per ciascuna coppia di sapore vero e predetto. I light-jet mostrano le prestazioni migliori, con un'efficienza del 96% e contaminazioni inferiori al 3% verso tutte le altre classi. I b-jet vengono identificati correttamente nell'87% dei casi; la perdita principale avviene verso i light-jet (11%). La classificazione dei τ -jet raggiunge un'efficienza del 58%, con contaminazione prevalente di light-jet (32%); tale

comportamento riflette la somiglianza tra i decadimenti adronici del τ e i jet leggeri, specialmente a basso p_T . La categoria più difficile rimane quella dei c-jet: l'efficienza è solo del 16%, con la maggioranza classificata come light-jet (53%). Il marcato sbilanciamento del dataset (Fig. 1b) contribuisce allo scarso risultato su τ - e c-jet, poiché il modello ottimizza implicitamente le prestazioni sulle classi più abbondanti.

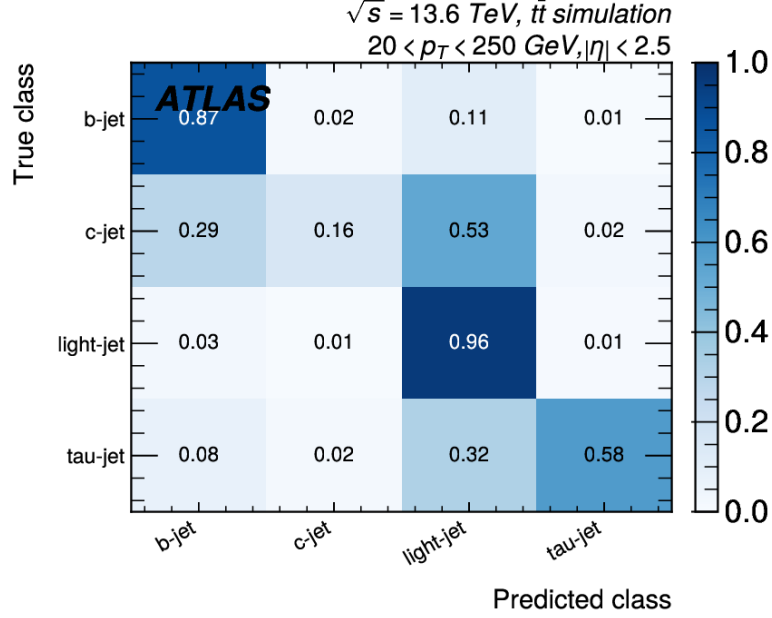


Figura 4: Matrice di confusione normalizzata sul test set.

6.3 Curve ROC

Le curve ROC mostrano il rigetto del fondo in funzione dell'efficienza di b - (Fig. 5) e c -tagging (Fig. 6). Le curve ROC per il discriminante D_c mostrano un regime di rigetto complessivamente più modesto, coerente con le difficoltà già evidenziate. Al 70% di efficienza di b -tagging il modello raggiunge rigetti di 700 (light-jet), 100 (τ -jet) e 12 (c-jet), rispetto a 2000, 400 e 50 di GN2 (fattore $\sim 3-4$). Al 30% di efficienza di c -tagging i rigetti scendono a 70 (light-jet), 12 (b-jet) e 4 (τ -jet), contro 300, 20 e 60 di GN2 (fattore $\sim 5-10$); il rigetto del τ -jet pari a 4 indica un'assenza quasi totale di discriminazione in questo regime.

6.4 Riepilogo

Nel complesso, il modello riproduce qualitativamente il comportamento atteso per un tagger basato su transformer: eccellente separazione dei light-jet, buone prestazioni per i b-jet, e difficoltà strutturale nel classificare c-jet e τ -jet. Le prestazioni quantitative sono inferiori al riferimento ATLAS di un fattore $\sim 3-4$ per il b -tagging e di un fattore $\sim 5-10$ per il c -tagging. Tali differenze sono ragionevoli considerando che questa implementazione utilizza un sottoinsieme del dataset originale, non applica ottimizzazione sistematica degli iperparametri, oltre ad avere solo una frazione di parametri rispetto a GN2.

Fondo	GN2 (ATLAS)	Questa impl.	Fondo	GN2 (ATLAS)	Questa impl.
Jet- c	50	12	Jet- b	20	12
Jet leggero	2000	700	Jet leggero	300	70
Jet- τ	400	100	Jet- τ	60	4

Tabella 3: Rigetto del fondo al 70% di efficienza di b -tagging (sinistra) e al 30% di efficienza di c -tagging (destra), confrontato con i valori ufficiali ATLAS per GN2.

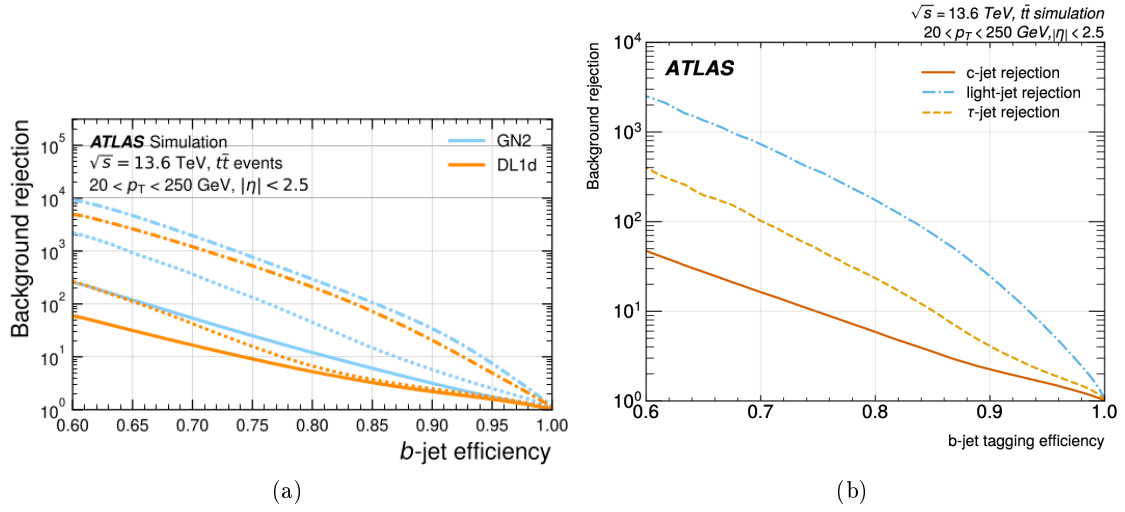


Figura 5: Curve ROC per il discriminante di b -tagging D_b : rigetto del fondo in funzione dell'efficienza di identificazione dei b -jet. A sinistra il risultato dell'articolo, mentre a destra il risultato con la nuova implementazione.

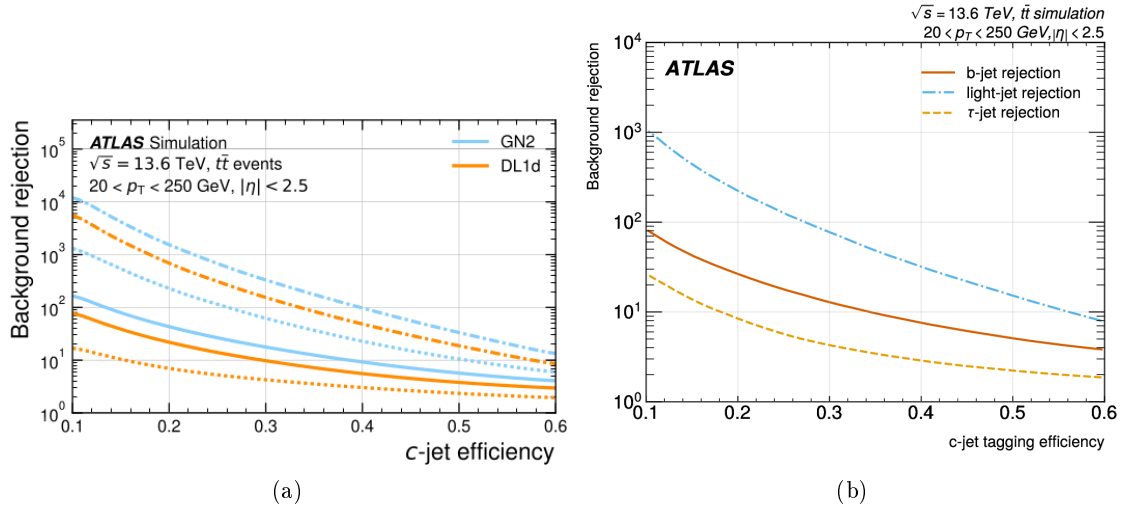


Figura 6: Curve ROC per il discriminante di c -tagging D_c : rigetto del fondo in funzione dell'efficienza di identificazione dei c -jet. A sinistra il risultato dell'articolo, mentre a destra il risultato con la nuova implementazione.